

Lab 2D: Ship It

Duration: 45 min

After: Git Workflow & Best Practices slides

Continues from: Lab 2C (your Business Dashboard should have separate route files and a task form)

Goal

Polish the dashboard into something professional, then use Git workflows in natural language. Start rough, iterate to great — this is how professionals use Claude Code every day.

Clear Context

Start fresh so you're not carrying Lab 2C's conversation history. Your project files and CLAUDE.md stay intact — this only clears the conversation.

```
/clear
```

Steps

1. Add Data Visualization

```
Add a simple bar chart to the dashboard showing task counts by status  
(pending, in-progress, completed). Use vanilla JavaScript and CSS –  
no chart libraries. Make it look good with our dark theme colors.
```

2. Verify in Browser

Open `http://localhost:3100` and look at the chart.

3. Critique and Refine

Look at the result, then tell Claude what to improve:

```
The chart works but [describe what you'd change – colors, spacing, labels, etc.]  
Make it better.
```

***This is the key skill.** You don't need to know how to fix it. You just need to know what "better" looks like and describe it. Claude handles the implementation.*

Try 2-3 rounds of refinement. Each time, describe what's not right and let Claude fix it.

4. Add Search and Filter

```
Add a filter bar above the tasks list:  
- Dropdown to filter by status (All, Pending, In Progress, Completed)  
- Dropdown to filter by priority (All, High, Medium, Low)  
- Filters should update the list immediately (no page reload)
```

Test the filters in your browser.

5. Production Readiness Review

```
Perform a production readiness review of this system.  
Identify issues in these categories:  
- security  
- scalability  
- reliability  
- observability  
- configuration management  
  
Rank the top 5 improvements required before production deployment.  
Explain why each one matters.
```

Review the assessment, then:

```
Implement the top 2 improvements while keeping the system fully functional.
```

You just used Claude as a technical collaborator — not just a code generator. Claude knows the difference between demo code and production code, and it can prioritize what matters most.

6. Commit Your Polish Work

```
Commit all improvements with message "feat: add chart, filters, and production hardening"
```

Security Hardening

7. Configure Permissions

You've been clicking "approve" all day. Now understand what you were approving:

```
/permissions
```

Review your current permission settings. Then configure smart defaults:

```
Set up permission rules for this project:  
- Auto-approve: file reads, grep, git status, git diff, npm test  
- Require approval: file writes, git commit, git push, npm install  
- Block: rm -rf, any command with sudo, access to ~/.ssh or ~/.aws  
Show me the .claude/settings.json configuration.
```

Test that the rules work:

```
Read server.js and show me the first 10 lines
```

That should auto-approve (read operation). Now:

```
Add a comment to the top of server.js
```

That should require your approval (write operation). **This is the trust level balance** — speed for safe operations, control for dangerous ones.

8. Security Audit

Ask Claude to be your security reviewer:

```
Perform a security audit of this entire project. Check for:
- Hardcoded secrets or API keys
- SQL injection vulnerabilities
- Missing input validation on any endpoint
- Improper error handling that leaks internal details
- Missing security headers (CORS, CSP, HSTS)
- Any dependencies with known vulnerabilities (run npm audit)
```

```
Rate each finding as CRITICAL, WARNING, or INFO.
Fix all CRITICAL findings immediately.
```

Review what Claude finds. Then verify the fixes:

```
Re-run the security audit. Have all CRITICAL issues been resolved?
```

9. Environment Configuration

Production code should never contain hardcoded values:

```
Create a .env.example file showing all environment variables this project needs:
- PORT, NODE_ENV, DATABASE_PATH
Then ensure server.js reads from environment variables with sensible defaults.
Add .env to .gitignore if it isn't already.
```

10. Commit Security Improvements

```
Commit all security improvements with message "feat: add permission rules, security
hardening, and env config"
```

The CISO pitch: Every AI code change requires explicit approval for writes. Read operations are fast. Dangerous commands are blocked. And you just ran a security audit in 60 seconds that would take a human reviewer 30 minutes.

Feature Branch Workflow

11. Create a Feature Branch

```
Create a new git branch called "feature/export" and switch to it
```

12. Build the Export Feature

```
Implement a CSV export feature with these requirements:  
- GET /api/export/metrics?format=csv – downloadable CSV of all metrics  
- GET /api/export/tasks?format=csv – downloadable CSV of all tasks  
- The CSV logic must live in a reusable module (src/utils/csvExport.js)  
- Routes should remain thin controllers  
- Add "Export CSV" buttons to the dashboard UI for both sections  
- Follow CLAUDE.md conventions
```

13. Test the Export

Ask Claude to verify:

```
Test both CSV export endpoints and show me the output
```

Also test the buttons in your browser — they should download .csv files.

14. Commit and Merge

```
Commit the export feature, switch back to main, and merge the feature branch
```

15. The Scaling Question

```
If this dashboard suddenly had 50,000 users:  
What parts of the system would break first?  
What architectural changes would you recommend?
```

Claude will explain SQLite limits, caching needs, pagination, async job queues. This is staff-engineer-level thinking from a single prompt.

16. Review Your Git History

```
Show me the git log with a nice one-line format
```

Expected output: Clean conventional commits telling the story of your project:

```
abc1234 feat: add CSV export for metrics and tasks
def5678 feat: add chart, filters, and production hardening
ghi9012 feat: add task creation form to frontend
jkl3456 refactor: extract routes into separate files
mno7890 feat: add task statistics endpoint
pqr1234 feat: initial business dashboard with API and UI
```

Optional: Go Deeper

The "interview me" pattern:

```
I want to add user authentication to this dashboard. Before writing any code,
interview me: ask me 5 questions about my requirements so you build exactly
what I need.
```

Answer Claude's questions, then let it build the auth system. Notice how much better the result is when Claude asks first.

Code review your own project:

```
Review the entire codebase against our CLAUDE.md conventions.
Are there any violations? List them and fix the top 3.
```

UI polish round:

```
Look at @public/index.html — the dashboard needs these improvements:
1. Add a last-updated timestamp that refreshes every 30 seconds
2. Metrics cards should show a colored indicator: green for positive trends, red for issues
3. The task list should show how long ago each task was created (e.g., "2 hours ago")
4. Add subtle hover effects on cards and task items
```

Add a completely new feature of YOUR choice:

Think about what would make this dashboard useful for YOUR business. Ask Claude to build it. Examples: - Email notifications when a high-priority task is created - A notes/comments field on each task - Metric history charts showing trends over time - User roles (admin vs viewer) - Dark/light theme toggle

□ Lab 2D Final Checkpoint

Your completed Business Dashboard includes: - ☐ REST API with 10+ endpoints - ☐ SQLite database (zero configuration) - ☐ Professional dark-themed web UI - ☐ Input validation with proper error responses - ☐ Bar chart visualization - ☐ Task filters (status + priority) - ☐ CSV export for metrics and tasks - ☐ Add-task form on the frontend - ☐ Permission rules configured (auto-approve reads, approve writes, block dangerous) - ☐ Security audit passed (no CRITICAL findings) - ☐ Environment configuration (.env.example, no hardcoded values) - ☐ Production hardening (security headers, logging) - ☐ CLAUDE.md with team conventions - ☐ Clean git history with conventional commits - ☐ Feature branch workflow

The Story of Your Codebase

One final prompt:

```
Summarize the entire evolution of this codebase. Explain:  
- what the system looked like at the beginning  
- each architectural change we made  
- how the system improved over time  
Use the git history and current codebase to tell the story.
```

Claude reconstructs the entire engineering journey — from empty folder to production-ready system. That's the mic-drop.

What You Built vs. What You Typed

Count your prompts. You probably typed **fewer than 25 messages** to Claude Code across all four labs and got:

- ~500 lines of backend code
- ~200 lines of frontend code
- A working SQLite database with sample data
- A professional UI with charts, filters, and forms
- Production-quality error handling and security
- Clean git history with 6+ conventional commits

That's the leverage. You described what you wanted. Claude Code built it.

Key Takeaways

Lab	Concept	Key Lesson
2A	The Agentic Loop	You provide intent, Claude provides implementation
2B	CLAUDE.md	One file makes conventions automatic — policy, not documentation
2C	@ Mentions + Recovery	Precision saves tokens; recovery prompting rebuilds what's broken
2D	Security + Ship	Permissions control the trust balance; iterate to production quality

☐ Continues in Day 3

Tomorrow, you'll take this exact dashboard and add: - Custom slash commands for common operations - Security hooks (secret detection, audit logging) - A report-generation skill - MCP integration for live data - Deployment with subagents working in parallel

Don't delete this project. You'll need it tomorrow.

Troubleshooting

Issue	Fix
Port 3100 in use	Tell Claude: "The port is in use, switch to 3200"
<code>better-sqlite3</code> won't install	Tell Claude: "better-sqlite3 failed to install, fix it"
Database locked	Tell Claude: "The database is locked, stop any running servers and restart"
Claude seems confused	Run <code>/clear</code> and re-state what you want with @ file references
Git conflicts	Tell Claude: "Resolve any git conflicts and show me the result"
Node version too old	Run <code>node -v</code> — must be 18+. Install latest from nodejs.org
Claude Code not found	See Pre-Flight section in Lab 2A for installation commands
Slow responses	Try <code>/model sonnet</code> for faster (but slightly less capable) responses
Context too large	Run <code>/compact</code> to compress conversation history
Claude not following conventions	Verify <code>CLAUDE.md</code> is in project root. Try <code>/clear</code> so Claude re-reads it
<code>tmpclaude-*</code> files in project	Temp files from Claude's editing process. Safe to delete

